

Seat 126: Our AI Math Checker

An easy-to-understand explanation of our Annapurna Labs Hackathon project, from the problem to what we built and tested.

1. What is our project about?

Imagine you ask an AI to solve a difficult math question. It gives you an answer that looks convincing - but what if the answer is wrong? Our project adds a mathematical checker, so the AI does not have to be trusted blindly.

Our big idea: Build an AI system that proposes a solution and then checks whether the solution is mathematically correct.

2. What math problems are we solving?

We work with **partial differential equations (PDEs)**. A simple example is a hot metal rod. Heat spreads from the hotter part to the colder part. A PDE can describe how the temperature at each point changes over time.

PDEs also appear in weather prediction, engineering, fluid movement, and other scientific problems.

3. What is the problem statement?

AI models can produce equations that look correct but do not actually satisfy the original problem. We asked: **Can automatic mathematical verification make an AI more reliable at solving PDEs?**

4. What did we build?

Our system has an AI solver and a math checker. The AI proposes a solution. The checker tests it. If a check fails, the system can provide feedback so the AI can try again.

1. PDE question

2. Qwen3-8B AI proposes a solution

3. SymPy verifies the solution

4. Accept if valid; otherwise give feedback and retry

Think of Qwen3-8B as the student, SymPy as the teacher, and our Python program as the classroom system connecting them.

5. What is AWS Trainium2?

AI models need powerful computing hardware. **AWS Trainium2** is Amazon's AI accelerator hardware. We used it to run the existing Qwen3-8B model during the hackathon. We did not invent the model or the chip; we built the workflow around them.

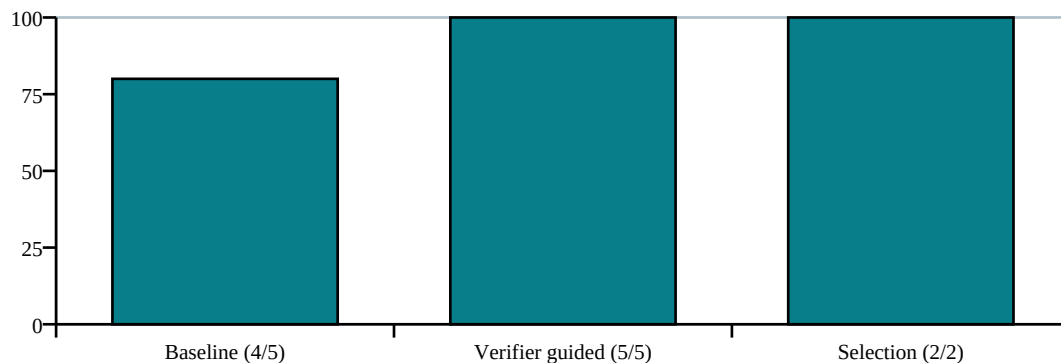
6. How does the checker decide whether an answer works?

For a heat-equation problem, the checker can test four things: whether the proposed solution satisfies the differential equation, matches the starting temperature, obeys the conditions at the ends of the rod, and can be handled as a valid symbolic expression.

If a check fails, that gives the agent a specific reason to revisit the answer.

7. What results did we get?

We ran a small set of benchmark cases. The baseline passed **4 out of 5**. The verifier-guided approach passed **5 out of 5**. A separate independent-selection experiment passed **2 out of 2**.



Graph shows the percentage of successful cases. These are **small tests**, not proof of perfect reliability. The 2/2 result comes from a separate experiment and should not be treated as an identical comparison.

8. What did our team actually do?

We ran Qwen3-8B on Trainium2, wrote Python code to send PDE questions to the model, created SymPy-based mathematical checks, connected those checks to an agent feedback workflow, ran comparison experiments, saved logs and results, and submitted our work through GitHub.

9. Why does this matter?

Scientific and engineering calculations need more than plausible-looking answers. A verification step can catch some mathematical errors before people rely on an AI result. Our project is an early prototype, not a fully validated engineering tool.

10. How do I explain it to judges in 30 seconds?

"Our project is a verifier-guided AI agent for solving partial differential equations. AI can generate mathematical answers that look right but are wrong. We ran Qwen3-8B on AWS Trainium2 and connected it to a SymPy mathematical verifier. The AI proposes a solution, the verifier checks the equation and its conditions, and failures can guide another attempt. In our small benchmark, the baseline passed four of five cases and the verifier-guided approach passed five of five."

Remember this: We built an AI that solves difficult math problems and uses a mathematical checker to verify its answers.